Competitive Programming Learning Resources

A structured path from ~1200 (Codeforces) upward, plus targeted references for XOR linear basis, DP optimization, and offline Fenwick-tree techniques.

---

1. General Roadmap (Intermediate, ~1200 CF rating)

Core curriculum

- **USACO Guide** — https://usaco.guide/ — the best structured curriculum overall. Bronze ≈ CF 800–1200, **Silver ≈ CF 1200–1600 (your target zone now)**, Gold ≈ CF 1600–2100. Each page has explanation + curated links + tagged practice problems.
- **CSES Problem Set** — https://cses.fi/problemset/ — ~300 problems organized by topic (sorting/searching, DP, graphs, etc.). No fluff; widely credited for 1200→1900 jumps.
- **Competitive Programmer's Handbook** (free PDF, by the CSES author) — https://cses.fi/book/book.pdf — concise theory companion to CSES.
- **CP-Algorithms** — https://cp-algorithms.com/ — the encyclopedia. Use as a reference/lookup, not a linear read.
- **errichto** YouTube channel — search "errichto DP", "errichto binary search" etc. Strong for DP, binary search, graphs when text isn't clicking.
- **Codeforces EDU** — https://codeforces.com/edu — structured video + problem courses (segment trees, DP optimizations). Good once Silver feels comfortable.

Core topics to master at this level

- Binary search on the answer ("can we achieve X?" framing, not just array search)
- Prefix sums / difference arrays (including 2D)
- Two pointers / sliding window
- Greedy proofs: exchange argument, interval scheduling
- DFS / BFS on graphs: grid BFS, connected components, bipartiteness
- DP fundamentals: 1D DP, knapsack variants, grid DP, small-N bitmask DP
- Sorting tricks: custom comparators, coordinate compression
- Basic number theory: GCD/LCM, modular arithmetic, sieve of Eratosthenes, fast exponentiation
- Fenwick Tree (BIT) and Segment Tree — essential once past ~1400

Practice strategy

- Solve by **rating + tag** on the Codeforces problemset filter (e.g. rating=1300–1400, tag=dp) — not just random Div 2 rounds.
- Hold Div 2 rounds as **virtual contests** (built into CF) under time pressure to build speed and instincts.
- **Upsolve everything** after each contest before moving to new problems — the single highest-leverage habit people report.

• Target problems rated **~100–200 above** your current rating most of the time.

• When stuck (after a genuine 30–45 min try): read just enough of the editorial to get the idea, implement it yourself, only read further if still stuck.

Suggested weekly rhythm

• 1 timed virtual Div 2 contest per week

• Upsolve everything from it

• 2–3 topic-based problems/week from CSES or the CF problemset, rotating through the topic list above

• 1 editorial deeply per week for something unsolved; re-implement it a few days later from memory (spaced repetition for patterns)

2. XOR Linear Basis (foundation for max-subset-XOR problems)

• **CP-Algorithms — Linear Basis** — https://cp-algorithms.com/linear_algebra/linear-basis.html — the best concise reference: building a basis, checking representability, max/min XOR subset, k-th XOR value.

• Search **"Codeforces XOR basis blog"** for community write-ups with pictures walking through greedy insertion and Gaussian-elimination-style reduction.

• **USACO Guide — XOR Basis** — https://usaco.guide/adv/xor-basis — has attached practice problems with editorials.

Practice problems

• **Codeforces 1100F "Ivan and Burgers"** — do this one specifically. It is the origin of the "position-priority basis" trick.

• Codeforces 895C "Square Subsets" — easier basis warm-up.

• SPOJ XMAX / "Maximum Xor Secondary" style problems for plain max-subset-XOR practice.

Position-priority / offline basis trick

• Read the **Codeforces 1100F editorial** (search "CF 1100F Ivan and Burgers editorial") — the canonical source of the "keep the newer-index vector at each pivot bit" trick. Read 2–3 people's solutions too, since editorials are terse.

• Search **"linear basis with greedy replacement by index"** or **"online/offline XOR basis for range queries"** for further CF/AtCoder discussion threads.

3. DP Over Partitions + Segment-Tree/Fenwick Optimization

• Search **"CF DP optimization segment tree"** for the general pattern dp[i] = max(dp[j] + cost(j,i)) computed via a data structure — sometimes called "1D/1D DP optimization."

• Codeforces EDU (by adamant/cdkrot) has a lecture titled **"DP optimizations"** covering monotonic stack, divide & conquer, Knuth's optimization, and segment-tree-based transitions.

• Practice: any "partition array into k segments minimizing/maximizing cost" problem on Codeforces/AtCoder — filter by tags **dp** + **data structures**.

4. Fenwick Tree & Offline Pair-Counting Pattern

Used for problems like: "count pairs i<j with f(i) ≤ g(j)" (decoupled two-index conditions).

- **Fenwick Tree / BIT basics** — CP-Algorithms: https://cp-algorithms.com/data_structures/fenwick.html
- The exact "count pairs i<j with f(i) ≤ g(j)" skeleton is the same one used for **counting inversions** — search "count inversions Fenwick tree tutorial".
- **USACO Guide — Point Update Range Sum** — https://usaco.guide/silver/PURS — covers BIT basics with this exact use case.
- Coordinate compression: short section on the same USACO Guide range-query pages; also search "coordinate compression competitive programming".

Practice problems

- Codeforces 1194E "Count Pairs" — near-identical skeleton, different algebraic transform.
- Codeforces 1311F, or any CF problem tagged "data structures" + "sortings" involving inversions.
- LeetCode 315 "Count of Smaller Numbers After Self" — smaller scale, exact same BIT + compression mechanic; good first hands-on implementation.

General tip: whenever a condition mixes two indices like expr(i, j) ≤ 0, try algebraically isolating all the i-only terms on one side and all the j-only terms on the other before reaching for a data structure. That decoupling habit, plus deliberate upsolving, is most of what separates 1200 from 1900.